CLAUDE CODE / 会话管理 / 1M 上下文

# 使用 Claude Code：会话管理与 1M 上下文

这篇文章的核心信息很简单：更好的结果来自于在恰当时机选择恰当的上下文控制方式。

继续 回退 压缩 清空 Subagent

# 基础概念：上下文既是能力来源，也会带来负担

窗口里都有什么

- 系统提示词、对话历史、工具调用、工具输出，以及文件读取记录
- Claude Code 现在提供 **100 万 token** 的上下文窗口
- 更长的任务成为可能，但注意力仍然需要被主动管理

上下文腐化

随着窗口变大，陈旧且无关的细节会争夺注意力，整体表现也会逐渐漂移。

压缩

接近上限时，Claude 会把当前会话总结成更短的交接摘要，并从该摘要继续。

**关键转变：**更大的窗口只是给你更多空间，并不意味着所有任务都可以永远塞在同一条线程里。

第 01 部分

# 每一轮回复都是一个分叉点

每次回答之后，都要决定是保留上下文、压缩它、重写它，还是把下一项任务隔离出去。

# 选项 1：当同一份上下文仍然关键时，继续

### 同一个任务

当前目标没有变化，前面的工作仍然直接相关。

### 已读取的文件仍有价值

Claude 已经加载了你接下来还需要的那些文件，重新开始只会把同样的状态再建一遍。

### 阻力最小

如果当前会话状态健康，继续就是最快、成本最低的推进方式。

> *Anthropic 的经验法则是：如果你真正开始了一个新任务，就同时开启一个新会话。否则，只有在当前上下文仍然有帮助时才继续。*

# 选项 2：与其在错误路径上纠缠，不如回退

在这些情况下使用 `/REWIND` 或双击 ESC

- Claude 探索了一条失败的方案
- 读取文件这一步是有用的，但尝试的修复方式不对
- 你希望从对话里一个更干净的分叉点重新提示

为什么有效

你可以保留好的准备工作，丢掉嘈杂的绕路过程，而不是在一个错误分支上不断叠加修正。

更好的提示方式：**“不要使用方案 A。这个模块并没有暴露那个 API。直接走方案 B。”**

# 选项 3：用 compact 还是 clear，取决于由谁决定保留什么

`/COMPACT`

## 由 Claude 编写摘要

- 更快，所需投入更少
- 适合在任务中途保持推进节奏
- 可以通过提示引导，比如保留认证部分、丢掉测试调试过程

`/CLEAR`

## 由你编写新的任务说明

- 最适合任务已经真正改变时使用
- 由你精确控制哪些约束和文件需要继续保留
- 更费力，但不会带上意外包袱

**Compact 保留连续性。 Clear 保留主动选择权。** 正确的选择取决于谁来决定下一步什么最重要。

第 02 部分

# 有意识地获得全新上下文

自动压缩可能会漏掉支线内容，所以要在高噪音工作污染主会话之前，主动把它隔离出去。

# 什么会导致糟糕的自动压缩?

1. 漫长的调试路径

整个会话被某一条调查主线主导。

2. 自动摘要

Claude 会压缩这条主线中看起来最相关的内容。

3. 任务切换

你的下一条提示突然跳到另一个警告或边缘问题。

失败模式

由于压缩发生时窗口往往已经很拥挤，优先级较低的细节就可能从摘要里掉出去。文章提到的正是这种模式：由于会话一直在关注别的事情，**bar.ts** 里的警告信息就消失了。

> *有了 1M 上下文，你就有更多时间提前执行压缩，并明确告诉 Claude 下一阶段应该保留什么。*

# Subagent 是处理高噪音工作的“洁净室”方案

在这些情况下使用 SUBAGENT

- 下一步会产生大量中间输出
- 你只需要把结论带回主会话
- 这项工作天然适合拆开，比如验证、代码库搜索、文档撰写

ANTHROPIC 的判断问题

我之后还需要这些工具输出吗，还是只需要结论?

如果答案只是结论，那就把它隔离到 subagent 里。

验证 | 跨仓库阅读 | 文档起草 | 高噪音搜索

# 决策表：什么时刻该用什么控制方式?

| 情境 | 优先选择 | 原因 |
| --- | --- | --- |
| 还是同一个任务，上下文仍然相关 | **Continue** | 不要为重建有用状态付出额外成本。 |
| Claude 走错了路径 | **Rewind** | 保留有价值的读取结果，丢掉失败分支。 |
| 任务进行中，会话已经臃肿 | **`/compact`** | 快速甩掉陈旧上下文，同时保持推进节奏。 |
| 你开始了一个新任务 | **`/clear`** | 清除上下文腐化，只保留你明确选择继续携带的内容。 |
| 下一步会很嘈杂，但你只关心结果 | **Subagent** | 把工具噪音隔离在主线程之外。 |

用一句话概括这篇文章：**管理的是上下文的形状，而不只是上下文的大小。**